Unit-2: Process and Threads Management

Main Topic	Subtopics	Focus for Notes
Process Concept	Definition of process, program vs process, process components	Basic concept and examples
Process States	New, Ready, Running, Waiting, Terminated states	Process state transition diagram
Process Control	Process Control Block (PCB), process creation and termination	Structure of PCB and functions
Threads	Concept of threads, benefits of threads	Difference between process and thread
CPU Scheduling	Need for scheduling, scheduling goals	Concept explanation
Types of Scheduling	Preemptive scheduling, Non-preemptive scheduling	Difference and examples
Scheduling Algorithms	FCFS, SJF, Round Robin, Priority scheduling	Working and comparison
Thread Scheduling	Scheduling of threads in OS	Basic understanding
Real-Time Scheduling	Concepts of real-time scheduling	Hard vs soft real-time scheduling
System Calls (Process Management)	ps, fork(), join(), exec(), wait()	Purpose and usage

Process Concept

1. Definition of Process : A **Process** is a program that is currently in execution.

A program stored on disk is a passive entity, but when it is loaded into memory and executed by the CPU, it becomes an active entity called a **process**.

Simple definition for exam

A process is an instance of a program in execution along with its current state, resources, and execution context.

Key Points

- A program becomes a **process when it starts executing**
- Multiple processes can run simultaneously in a system
- Each process has its own **memory space and resources**
- The operating system manages processes using **process management techniques**

Basic Structure of a Process

A process contains several components required for execution.

Component	Description
Program Code	Instructions to be executed
Program Counter	Address of next instruction
Stack	Function calls and local variables
Heap	Dynamically allocated memory
Data Section	Global variables

Simple Process Structure Diagram

Process

Program Code (Text Section)
Data Section
Heap
Stack
Program Counter
CPU Registers

Program vs Process

A program and a process are closely related but different concepts.

Feature	Program	Process
Definition	Set of instructions stored on disk	Program in execution
Nature	Passive entity	Active entity
Location	Stored in secondary memory	Loaded into main memory
Execution	Cannot execute itself	Executes using CPU
Example	.exe file	Running application

Example:

Program	Process
Chrome.exe file	Chrome running in RAM
Notepad application file	Notepad running on screen

Characteristics of a Process

Characteristic	Description
Dynamic	Created during execution
Independent	Each process runs independently
Concurrent	Multiple processes can run simultaneously
Resource ownership	Each process uses CPU, memory, and I/O
State changes	Process changes states during execution

Components of a Process

A process consists of multiple memory sections.

1. Text Section

Contains the **compiled program code**.

2. Data Section

Stores **global and static variables**.

3. Stack

Used for:

- function calls
- local variables
- return addresses

4. Heap

Used for **dynamic memory allocation** during execution.

Example of Process Execution

Consider a program named **Calculator**.

Steps:

1. Program stored in disk
2. User runs the program

3. OS loads program into RAM
4. CPU begins executing instructions
5. The program becomes a **process**

Flow:

Program on Disk
↓
Loaded into Memory
↓
CPU Executes Instructions
↓
Process Created

Role of Operating System in Process Management

The operating system performs several tasks related to processes.

Task	Description
Process Creation	Creates new processes
Process Scheduling	Decides which process runs
Process Synchronization	Coordinates processes
Process Termination	Stops processes after completion

Example of Processes in a System

A computer may run multiple processes simultaneously.

Examples:

Process	Purpose
Web Browser	Internet browsing
Music Player	Playing audio
Code Editor	Writing programs
File Manager	Managing files

The operating system schedules these processes so that they appear to run simultaneously.

Simple Diagram of Multiple Processes

Operating System
↓

Process 1 → Web Browser
Process 2 → Music Player
Process 3 → Code Editor
Process 4 → File Manager

↓
CPU executes processes one by one

Short Conclusion: A **process** is a program that is currently executing in memory. It contains program code, data, stack, and execution state. The operating system manages processes by scheduling CPU time, allocating resources, and controlling execution.

Process States

1. Definition: A **Process State** represents the current status of a process during its execution in the operating system.
While executing, a process does not remain in one condition. It continuously changes its state depending on CPU availability, input/output operations, or completion of execution.
The operating system manages these states to control the execution of processes efficiently.

Main Process States
A typical operating system defines **five basic states of a process**.

State	Description
New	Process is being created
Ready	Process is ready to run and waiting for CPU
Running	Process is currently executing on the CPU
Waiting (Blocked)	Process is waiting for an event or I/O operation
Terminated	Process has completed execution

Explanation of Each Process State
1. New State
When a program is first loaded into memory, the operating system creates a **process**.
At this stage:

- The process is being initialized
- Resources such as memory and process control block are allocated

Example:
User starts a program like **Notepad**.

2. Ready State: After creation, the process enters the **ready queue**.
Characteristics:

- Process has all required resources
- Waiting for CPU allocation
- Ready to execute

The OS scheduler decides which process will run next.
Example:

Process	Status
Process A	Ready
Process B	Ready
Process C	Ready

Only one will be selected for execution.

3. Running State
In this state:

- The process is **currently executing instructions**
- The CPU is allocated to that process

Only **one process per CPU core** can be in the running state at a time.
Example:
CPU executing:
Process → Web Browser

4. Waiting (Blocked) State: A process enters the **waiting state** when it requires an external event.
Common reasons:

- Input/output operations
- Waiting for data
- Waiting for a resource

Example:
Process requests **disk data**.
Process → Waiting for disk read.

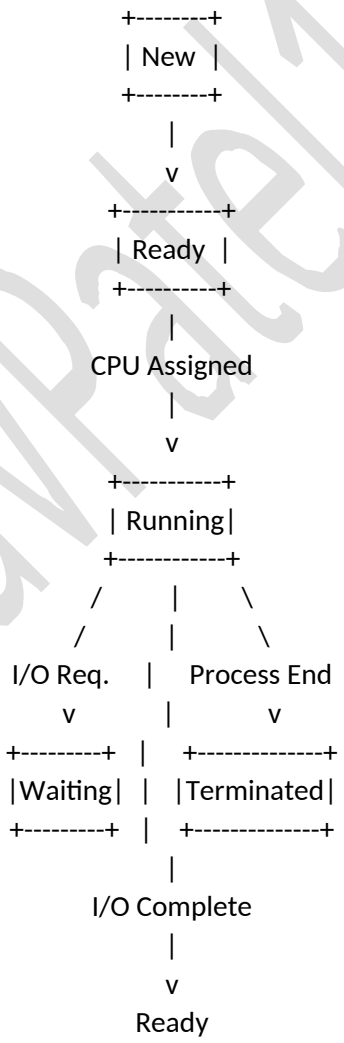
5. Terminated State
This is the **final state of a process**.
The process enters this state when:

- Program execution completes
- Process is terminated by OS
- Process encounters an error

After termination:

- OS releases all allocated resources.

Process State Transition Diagram
This diagram shows how a process moves between states.



Process Life Cycle

Definition: The **Process Life Cycle** describes the sequence of states through which a process passes from its creation to its termination.
It represents the **entire life span of a process in the operating system**.

Stages in Process Life Cycle

Stage	Description
Process Creation	OS creates the process
Ready Queue	Process waits for CPU
Process Execution	CPU executes process instructions
Waiting State	Process waits for I/O or events
Process Termination	Process finishes execution

Process Life Cycle Flow

Process Creation
↓
Ready
↓
Running
↓
Waiting
↓
Ready
↓
Running
↓
Terminated

The process may **repeat Ready → Running → Waiting multiple times** before termination.

Example of Process Life Cycle
Consider a program **Video Player**.
Step-by-step execution:

1. User opens video player → **New state**
2. OS loads program → **Ready state**
3. CPU executes video player → **Running state**
4. Video file loading → **Waiting state**
5. Data received → **Ready state**
6. Continue execution → **Running state**
7. User closes player → **Terminated state**

Role of Operating System in Process States
The operating system controls the process life cycle using:

Function	Description
Process Scheduling	Selects process for execution
Context Switching	Switches CPU between processes
Resource Allocation	Assigns CPU and memory
Process Termination	Releases resources

Key Points for Exams

- A process changes states during execution
- Five main states exist in most operating systems
- The OS scheduler controls transitions between states
- The process life cycle ends in the **terminated state**

Short Exam Definition (3–4 Marks)
Process states represent the different conditions that a process goes through during execution. A typical operating system defines five states: new, ready, running, waiting, and terminated. The transitions between these states represent the process life cycle, which begins when the process is created and ends when it finishes execution.

Process Control Block (PCB)
1. Definition : A **Process Control Block (PCB)** is a data structure used by the operating system to store all important information about a process.
It acts as a **process record** that contains details required by the operating system to manage and control the execution of the process.
Simple exam definition
A Process Control Block is a data structure maintained by the operating system that contains information about a process such as its state, program counter, registers, and scheduling information.

Purpose of PCB
The PCB helps the operating system:
- Track the **current state of a process**
- Manage **process scheduling**
- Perform **context switching**
- Store **process-related information**

Without PCB, the OS cannot manage multiple processes efficiently.

Components of Process Control Block
A PCB stores several types of information related to a process.

PCB Field	Description
Process ID (PID)	Unique identifier of the process
Process State	Current state (new, ready, running, waiting, terminated)
Program Counter	Address of next instruction to execute
CPU Registers	Values of CPU registers during execution
CPU Scheduling Information	Priority, scheduling queue
Memory Management Information	Base and limit registers, memory allocation
Accounting Information	CPU time used, process number
I/O Status Information	List of I/O devices allocated

Structure of Process Control Block

Process Control Block

Process ID
Process State
Program Counter

CPU Registers
CPU Scheduling Information
Memory Management Information
Accounting Information
I/O Status Information

The operating system maintains **one PCB for every process** in the system.

Explanation of Important PCB Fields
1. Process ID (PID): Each process has a **unique identification number**.
Example:

Process	**PID**
Browser	105
Music Player	220
Code Editor	310

This helps the OS distinguish between processes.

2. Process State
Stores the current state of the process.
Possible states:

- New
- Ready
- Running
- Waiting
- Terminated

The OS uses this information to manage process scheduling.

3. Program Counter
The program counter contains the **address of the next instruction** to be executed by the process.
During context switching, the OS saves this value inside the PCB.

4. CPU Registers
Stores the values of registers such as:

- Accumulator
- Index registers
- Stack pointer
- General-purpose registers

These values are restored when the process resumes execution.

5. CPU Scheduling Information
Contains scheduling data such as:

- Process priority
- Scheduling queue position
- CPU time allocated

This information helps the scheduler decide which process runs next.

6. Memory Management Information
Stores details about memory allocation.
Examples:

- Base register
- Limit register
- Page tables
- Segment tables

These are used to manage the memory space of the process.

7. Accounting Information

Contains information used for **process monitoring and statistics**.

Examples:

- CPU usage time
- Process execution time
- User ID

8. I/O Status Information

Stores information about input/output devices used by the process.

Examples:

- Open files
- Allocated devices
- Pending I/O requests

Role of PCB in Context Switching

When the CPU switches from one process to another, the OS performs **context switching**.

Steps:

1. OS saves the current process state into its PCB.
2. OS loads the PCB of the next process.
3. CPU resumes execution of the new process.

Flow:

Process A Running
↓
Save State in PCB-A
↓
Load PCB-B
↓
Process B Running

Example of PCB Usage

Suppose three processes are running:

Process	PID	State
Process A	101	Running
Process B	102	Ready
Process C	103	Waiting

The operating system stores all this information in their respective **PCBs**.

Advantages of PCB

Advantage	Explanation
Efficient process management	OS tracks all process information
Supports multitasking	Multiple processes handled easily
Context switching	Enables CPU switching between processes

Process monitoring	Allows tracking of resource usage

Short Conclusion

A **Process Control Block (PCB)** is a data structure maintained by the operating system that stores all information about a process. It includes process state, program counter, CPU registers, scheduling information, and memory details. The PCB allows the operating system to manage processes and perform context switching efficiently.

Process Scheduling (CPU Scheduling Concept)

1. Definition : Process Scheduling is the method used by the operating system to decide which process should use the CPU at a particular time.

Since multiple processes may be ready to execute at the same time, the operating system selects one process from the **ready queue** and allocates the CPU to it.

Simple exam definition : CPU scheduling is the process of selecting one process from the ready queue and allocating the CPU to it for execution.

Why CPU Scheduling is Needed

In a multiprogramming system, many processes are present in memory simultaneously. Only one process can execute on the CPU at a time.

CPU scheduling helps:

Purpose	Explanation
Efficient CPU utilization	CPU should not remain idle
Fair process execution	All processes get CPU time
Improved system performance	Reduces waiting time
Better response time	Important for interactive systems

Basic Concept of CPU Scheduling

When a process is created, it enters the **ready queue**. The operating system scheduler selects one process from this queue to run on the CPU.

Flow:

Processes in Ready Queue
↓
CPU Scheduler
↓
Selected Process
↓
CPU

Scheduling Criteria

CPU scheduling algorithms aim to optimize system performance.

Criteria	Meaning
CPU Utilization	Keep CPU as busy as possible
Throughput	Number of processes completed per unit time
Turnaround Time	Time from submission to completion
Waiting Time	Time spent waiting in ready queue
Response Time	Time taken to respond to a request

Types of CPU Scheduling

There are two main types of CPU scheduling.

Type	Description
Preemptive Scheduling	OS can interrupt a running process
Non-Preemptive Scheduling	Process runs until completion or blocking

Example:

Algorithm	Type
FCFS	Non-Preemptive
SJF	Both
Round Robin	Preemptive
Priority Scheduling	Both

First Come First Serve (FCFS) Scheduling

1. Definition : First Come First Serve (FCFS) is the simplest CPU scheduling algorithm in which the process that arrives first gets executed first.

It follows the principle:

First process to arrive → First process to execute

FCFS is a **non-preemptive scheduling algorithm**.

Working of FCFS Scheduling

Steps:

1. Processes enter the ready queue.
2. The process that arrives first gets the CPU first.
3. Once a process starts execution, it continues until it finishes.
4. The next process in the queue is executed.

Example of FCFS Scheduling

Suppose four processes arrive in the following order.

Process	Burst Time
P1	5
P2	3
P3	8
P4	6

Execution order will be:

P1 → P2 → P3 → P4

Gantt Chart Representation

| P1 | P2 | P3 | P4 |

0 5 8 16 22

Explanation:

Process	Start Time	Finish Time
P1	0	5
P2	5	8
P3	8	16
P4	16	22

Advantages of FCFS

Advantage	Explanation

Simple algorithm	Easy to implement
Fair scheduling	Processes executed in order of arrival
Low overhead	Minimal scheduling complexity

Disadvantages of FCFS

Disadvantage	Explanation
Convoy effect	Short processes wait for long processes
High waiting time	Small jobs may wait too long
Poor response time	Not suitable for interactive systems

Convoy Effect

In FCFS, if a **long process arrives first**, all short processes must wait behind it.

Example:

Process	Burst Time
P1	20
P2	2
P3	3

Execution:

P1 → P2 → P3

Here P2 and P3 must wait a long time.

Simple FCFS Diagram

Ready Queue

P1 → P2 → P3 → P4

↓

CPU

Short Conclusion

CPU scheduling is a mechanism used by the operating system to allocate CPU time to processes. The **First Come First Serve (FCFS)** algorithm schedules processes in the order they arrive in the ready queue. Although simple and easy to implement, FCFS may cause high waiting time due to the convoy effect.

Shortest Job First (SJF) Scheduling

1. Definition : Shortest Job First (SJF) is a CPU scheduling algorithm in which the process with the **smallest burst time (execution time)** is selected for execution first.

In this algorithm, the operating system always chooses the process that requires the **least CPU time** among all the processes in the ready queue.

Simple exam definition : Shortest Job First scheduling selects the process with the smallest CPU burst time for execution.

Basic Idea of SJF

The main goal of SJF is to **minimize the average waiting time** of processes.

Shorter processes finish earlier, which improves system performance.

Example idea:

Process	Burst Time

P1	6
P2	2
P3	8
P4	3

Execution order:
P2 → P4 → P1 → P3

Types of SJF Scheduling

There are two types of SJF scheduling.

Type	Description
Non-Preemptive SJF	Once a process starts execution, it runs until completion
Preemptive SJF	A new shorter process can interrupt the running process

Preemptive SJF is also called **Shortest Remaining Time First (SRTF)**.

Working of Non-Preemptive SJF

Steps:

1. All processes enter the ready queue.
2. The OS checks the burst time of each process.
3. The process with the **shortest burst time** is selected.
4. That process executes until completion.
5. The next shortest process is selected.

Example of SJF Scheduling

Suppose four processes arrive at the same time.

Process	Burst Time
P1	6
P2	8
P3	7
P4	3

Execution order based on shortest burst time:
P4 → P1 → P3 → P2

Gantt Chart Representation

| P4 | P1 | P3 | P2 |
0 3 9 16 24

Explanation:

Process	Start Time	Finish Time
P4	0	3
P1	3	9
P3	9	16
P2	16	24

Advantages of SJF

Advantage	Explanation
Minimum waiting time	Optimal scheduling algorithm
Better CPU utilization	Short tasks finish quickly
Efficient performance	Improves system throughput

Disadvantages of SJF

Disadvantage	Explanation
Difficult to predict burst time	Future execution time is unknown
Starvation problem	Long processes may wait indefinitely
Complex implementation	Requires burst time estimation

Starvation Problem

In SJF scheduling, long processes may wait for a very long time if shorter processes keep arriving.

Example:

Process	Burst Time
P1	20
P2	2
P3	3
P4	1

Execution:

P4 → P2 → P3 → P1

Process **P1 waits a long time**.

Simple Diagram of SJF

Ready Queue

P1(6) P2(2) P3(8) P4(3)

↓

Select process with smallest burst time

↓

CPU Execution

Comparison: FCFS vs SJF

Feature	FCFS	SJF
Selection	Arrival order	Shortest burst time
Waiting time	High	Minimum
Complexity	Simple	More complex
Starvation	Rare	Possible

Short Conclusion

Shortest Job First (SJF) is a CPU scheduling algorithm that selects the process with the smallest burst time for execution. It minimizes average waiting time and improves system efficiency, but it may cause starvation for longer processes and requires estimation of burst times.

Round Robin (RR) Scheduling

1. Definition

Round Robin (RR) is a CPU scheduling algorithm in which each process is assigned a fixed amount of CPU time called a **time quantum** or **time slice**.

Each process gets the CPU for a limited time. If the process is not finished within that time, it is **moved back to the end of the ready queue**, and the next process gets the CPU.

Simple exam definition

Round Robin scheduling is a preemptive CPU scheduling algorithm in which each process receives a fixed time slice of CPU time in cyclic order.

Basic Idea of Round Robin

Round Robin scheduling works like **a circular queue**.

Each process:

- gets CPU for a fixed time quantum
- if finished → process terminates
- if not finished → goes back to ready queue

This continues until all processes finish.

Time Quantum (Time Slice)

Time Quantum is the maximum time a process can run before it is interrupted.

Example:

Time Quantum	Meaning
2 ms	Each process runs for 2 milliseconds
5 ms	Each process runs for 5 milliseconds

If a process needs **more time than the quantum**, it is preempted.

Working of Round Robin Scheduling

Steps:

1. Processes enter the ready queue.
2. The first process gets CPU for **time quantum**.
3. If the process finishes → it terminates.
4. If not finished → it is placed at the end of the queue.
5. The next process gets CPU.
6. The cycle continues.

Example of Round Robin Scheduling

Suppose four processes exist.

Process	Burst Time
P1	5
P2	4
P3	2
P4	1

Time Quantum = **2**

Step-by-Step Execution

1st cycle:

P1 → 2 units

P2 → 2 units

P3 → 2 units (completed)

P4 → 1 unit (completed)

2nd cycle:

P1 → 2 units

P2 → 2 units (completed)

3rd cycle:
P1 → 1 unit (completed)

Gantt Chart Representation

| P1 | P2 | P3 | P4 | P1 | P2 | P1 |
0 2 4 6 7 9 11 12

Ready Queue Representation

Initial Ready Queue

P1 → P2 → P3 → P4

After first cycle

P1 → P2

Advantages of Round Robin

Advantage	Explanation
Fair scheduling	Each process gets equal CPU time
Good response time	Suitable for interactive systems
No starvation	All processes get CPU opportunity

Disadvantages of Round Robin

Disadvantage	Explanation
Context switching overhead	Frequent switching between processes
Performance depends on quantum	Too small quantum reduces efficiency
Higher waiting time	Compared to SJF

Effect of Time Quantum

The size of the time quantum significantly affects system performance.

Time Quantum	Effect
Very small	Too many context switches
Very large	Behaves like FCFS
Optimal size	Balanced performance

Simple Round Robin Diagram

Ready Queue

P1 → P2 → P3 → P4 → P1 → P2 ...

↓
CPU

(Time quantum assigned to each process)

Comparison of Scheduling Algorithms

Algorithm	Type	Key Idea
FCFS	Non-preemptive	First process executed first
SJF	Preemptive/Non-preemptive	Shortest job executed first
Round Robin	Preemptive	Each process gets equal CPU time

Short Conclusion

Round Robin is a preemptive CPU scheduling algorithm that allocates a fixed time slice to each process in cyclic order. If a process does not finish within its time quantum, it is moved to the end of the ready queue. This algorithm provides fairness and good response time, making it suitable for time-sharing systems.

Shortest Remaining Time Next (SRTN)

1. Definition: Shortest Remaining Time Next (SRTN) is a **preemptive CPU scheduling algorithm** in which the process with the **smallest remaining burst time** is selected for execution.
It is the **preemptive version of the Shortest Job First (SJF)** scheduling algorithm.
If a new process arrives with a **shorter remaining time than the currently running process**, the CPU switches to the new process.
Simple exam definition: SRTN scheduling selects the process with the shortest remaining execution time, and it can interrupt the currently running process if a shorter job arrives.

Basic Idea of SRTN

The scheduler always checks the **remaining execution time** of all processes.
The process with the **minimum remaining time** gets the CPU.
If a new process arrives with a **smaller burst time**, it immediately preempts the current process.

Working of SRTN Scheduling

Steps:

1. Processes arrive in the ready queue.
2. The scheduler selects the process with the **shortest remaining time**.
3. If a new process arrives with **smaller burst time**, the current process is interrupted.
4. The CPU switches to the new process.
5. This continues until all processes complete execution.

Example of SRTN Scheduling

Consider the following processes.

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	2
P4	3	1

Step-by-Step Execution

1. At time **0**, P1 starts executing.
2. At time **1**, P2 arrives (remaining time smaller than P1) → switch to P2.
3. At time **2**, P3 arrives (smaller burst time) → switch to P3.
4. At time **3**, P4 arrives → shortest job → executed first.

Gantt Chart Representation

| P1 | P2 | P3 | P4 | P3 | P2 | P1 |
0 1 2 3 4 5 7 14

Characteristics of SRTN

Feature	Description
Type	Preemptive scheduling
Selection	Shortest remaining time
Efficiency	Minimizes waiting time
Interruptions	Frequent preemption possible

Advantages of SRTN

Advantage	Explanation
Minimum average waiting time	Optimal scheduling algorithm
Fast response	Short processes finish quickly
Better system performance	Improves throughput

Disadvantages of SRTN

Disadvantage	Explanation
High context switching	Frequent process interruption
Starvation	Long processes may wait indefinitely
Complex implementation	Requires continuous monitoring

Starvation Problem

If many short processes keep arriving, longer processes may wait for a very long time.

Example:

Process	Burst Time
P1	20
P2	2
P3	1
P4	2

Execution order will prioritize shorter jobs, causing **P1 to wait longer**.

Simple SRTN Diagram

Ready Queue

P1(8) P2(4) P3(2) P4(1)

↓
Select process with smallest
remaining execution time
↓
CPU Execution

Difference Between SJF and SRTN

Feature	SJF	SRTN
Type	Non-preemptive	Preemptive

Process interruption	Not allowed	Allowed
Selection	Shortest burst time	Shortest remaining time
Complexity	Simple	More complex

Short Conclusion

Shortest Remaining Time Next (SRTN) is a preemptive scheduling algorithm that always executes the process with the smallest remaining burst time. If a new process arrives with a shorter remaining time than the currently running process, the CPU switches to the new process. Although it minimizes average waiting time, it may cause starvation and increased context switching.

Priority Scheduling (Preemptive Priority)

1. Definition : Priority Scheduling is a CPU scheduling algorithm in which each process is assigned a **priority value**, and the process with the **highest priority** gets the CPU first.
In **Preemptive Priority Scheduling**, if a new process arrives with a **higher priority than the currently running process**, the CPU is immediately assigned to the new process.
Simple exam definition : Preemptive Priority Scheduling is a scheduling algorithm in which the CPU is allocated to the process with the highest priority, and a running process can be interrupted if a higher priority process arrives.

Basic Idea of Priority Scheduling

Each process has a priority number.

- **Higher priority process → executed first**
- **Lower priority process → waits in ready queue**

Example priority rule:

Priority Number	Meaning
1	Highest priority
5	Lowest priority

(Some systems use the opposite convention.)

Working of Preemptive Priority Scheduling

Steps:

1. Each process is assigned a priority.
2. The scheduler selects the process with the **highest priority**.
3. The process starts executing.
4. If a new process arrives with **higher priority**, the running process is **preempted**.
5. The CPU is allocated to the higher priority process.

Example of Priority Scheduling

Consider the following processes.

Process	Arrival Time	Burst Time	Priority
P1	0	8	3
P2	1	4	1
P3	2	5	2
P4	3	3	4

(Assume **smaller number = higher priority**)

Step-by-Step Execution

1. At time **0**, P1 starts executing.

2. At time **1**, P2 arrives with higher priority → **P1 is preempted**.
3. P2 executes first.
4. After P2 finishes, the next highest priority process is selected.

Gantt Chart Representation

| P1 | P2 | P3 | P1 | P4 |
0 1 5 10 17 20

Characteristics of Priority Scheduling

Feature	Description
Type	Preemptive scheduling
Selection	Based on process priority
Flexibility	Important processes execute first
Control	Allows system control over execution order

Advantages of Priority Scheduling

Advantage	Explanation
Important tasks executed first	High priority processes run immediately
Flexible scheduling	Priority levels control execution
Useful for real-time systems	Critical processes can run earlier

Disadvantages of Priority Scheduling

Disadvantage	Explanation
Starvation problem	Low priority processes may never execute
Complex scheduling	Requires priority management
Unfair execution	Lower priority processes may wait indefinitely

Starvation Problem

In priority scheduling, **low priority processes may never get CPU time** if high priority processes keep arriving.

Example:

Process	Priority
P1	High
P2	Medium
P3	Low

If high priority processes continuously arrive, **P3 may wait indefinitely**.

Solution to Starvation: Aging

Aging is a technique used to gradually **increase the priority of waiting processes**.

Example:

Waiting Time	Priority Increase
5 seconds	Priority increases
10 seconds	Priority increases further

This ensures that **every process eventually gets CPU time**.

Simple Priority Scheduling Diagram

Ready Queue

P1(3) P2(1) P3(2) P4(4)

↓
Select process with
highest priority
↓
CPU Execution

Comparison of Scheduling Algorithms

Algorithm	Type	Selection Method
FCFS	Non-preemptive	Arrival order
SJF	Non-preemptive	Shortest burst time
SRTN	Preemptive	Shortest remaining time
Round Robin	Preemptive	Time quantum
Priority	Preemptive / Non-preemptive	Highest priority

Short Conclusion

Priority Scheduling assigns CPU time based on the priority of processes. In the preemptive version, if a higher priority process arrives, it immediately interrupts the currently running process. Although this algorithm allows important processes to execute first, it may cause starvation for low-priority processes, which can be solved using aging.

Real-Time Operating System (RTOS)

1. Definition: A **Real-Time Operating System (RTOS)** is an operating system designed to process data and provide results within a **strict time limit**.
In real-time systems, the **correctness of the system depends not only on the logical result but also on the time at which the result is produced**.
Simple exam definition : A Real-Time Operating System is an operating system that guarantees response to events within a fixed and predictable time constraint.

Basic Idea of Real-Time Systems

In many systems, tasks must be completed **within a specific deadline**.
If the system fails to respond within the required time, it may lead to **system failure or dangerous consequences**.
Examples:

System	Requirement
Aircraft control	Immediate response
Medical monitoring	Real-time patient data processing
Industrial robots	Precise timing control
Traffic signal systems	Accurate timing operations

Characteristics of Real-Time Operating System

Feature	Description
Deterministic behavior	Predictable execution time
Fast response	Quick reaction to external events
High reliability	System must operate without failure
Priority-based scheduling	Critical tasks executed first
Minimal delay	Very low latency in task processing

Types of Real-Time Operating Systems

Real-time operating systems are classified into two types.

1. Hard Real-Time System

In **Hard Real-Time Systems**, meeting the deadline is **mandatory**.

Missing a deadline may cause **system failure or serious damage**.

Examples:

Application	Example
Aircraft control systems	Flight control computers
Medical devices	Pacemakers
Industrial automation	Nuclear power plant control

2. Soft Real-Time System

In **Soft Real-Time Systems**, missing a deadline **does not cause system failure**, but performance may degrade.

Examples:

Application	Example
Multimedia streaming	Video playback
Online gaming	Network response
Virtual reality systems	Rendering delays

Working of Real-Time Operating System

Steps:

1. External event occurs (sensor input or signal).
2. The system receives the event.
3. RTOS scheduler selects the highest priority task.
4. The task is executed immediately.
5. The system produces output within the deadline.

Flow:

External Event

↓

Interrupt Signal

↓

RTOS Scheduler

↓

Task Execution

↓

Immediate Response

Advantages of RTOS

Advantage	Explanation
Fast response time	Immediate reaction to events
High reliability	Suitable for critical systems
Deterministic behavior	Predictable task execution
Efficient task management	Priority-based scheduling

Disadvantages of RTOS

Disadvantage	Explanation

Complex design	Requires specialized algorithms
Higher cost	Requires reliable hardware
Limited flexibility	Designed for specific tasks
Development difficulty	Programming is more complex

Examples of Real-Time Operating Systems

RTOS	Application
VxWorks	Aerospace and defense systems
QNX	Automotive systems
FreeRTOS	Embedded systems
RTLinux	Real-time embedded applications

Simple Diagram of RTOS

Sensors / External Events
↓
Real-Time Operating System
↓
Task Scheduling
↓
Immediate Output

Key Points for Exams

- RTOS guarantees **timely response to events**
- Used in **critical and time-sensitive applications**
- Two types: **Hard real-time and Soft real-time**
- Uses **priority-based scheduling**

Short Conclusion

A Real-Time Operating System is designed to process data and produce results within strict time limits. It ensures deterministic behavior and quick response to external events. RTOS is widely used in critical applications such as aerospace systems, medical devices, and industrial automation.

Threads

1. Definition : A **Thread** is the smallest unit of execution within a process.
A process can contain multiple threads that execute tasks concurrently while sharing the same memory and resources.
Simple exam definition : A thread is a lightweight process that represents a single sequence of execution within a program.

Basic Idea of Threads

In a traditional process, only one task runs at a time.
With threads, a single process can perform **multiple tasks simultaneously**.
Example:
A web browser may perform multiple operations at the same time.

Thread	Task
Thread 1	Loading webpage
Thread 2	Playing video
Thread 3	Downloading files

Thread 4	Handling user input

All these threads belong to the **same process**.

Components of a Thread

Each thread contains its own execution information.

Component	Description
Thread ID	Unique identifier of the thread
Program Counter	Address of next instruction
Register Set	CPU register values
Stack	Local variables and function calls

Threads share some resources of the process.

Shared Resources
Code section
Data section
Heap
Open files

Structure of Threads within a Process

Process

Code Section
Data Section
Heap

Thread 1 → Stack + Registers
Thread 2 → Stack + Registers
Thread 3 → Stack + Registers

All threads share the same memory but have separate stacks.

Types of Threads

Threads are classified into two main types.

1. User-Level Threads

User-level threads are managed by **user-level libraries** and not directly by the operating system.

Characteristics:

Feature	Description
Managed by	User-level thread library
Kernel awareness	OS does not know about threads
Performance	Fast thread switching

Examples:

- POSIX threads
- Java threads
- Thread libraries

2. Kernel-Level Threads

Kernel-level threads are managed directly by the **operating system kernel**.

Characteristics:

Feature	Description
Managed by	Operating system
Kernel awareness	OS manages thread scheduling
Performance	Slower due to system calls

Examples:

- Windows threads
- Linux kernel threads

Advantages of Threads

Advantage	Explanation
Improved performance	Multiple tasks executed simultaneously
Better CPU utilization	CPU remains active
Resource sharing	Threads share memory and data
Faster execution	Thread creation is faster than process creation

Disadvantages of Threads

Disadvantage	Explanation
Synchronization problems	Multiple threads accessing shared data
Complex programming	Difficult debugging
Security issues	Errors in one thread affect others

Process vs Thread

Feature	Process	Thread
Definition	Program in execution	Smallest unit of execution
Memory	Separate memory space	Shared memory
Creation time	Slow	Faster
Communication	Inter-process communication required	Direct communication possible
Overhead	High	Low

Example of Multithreading

Example: Web browser
Web Browser Process

Thread 1 → Page rendering
Thread 2 → Image loading
Thread 3 → JavaScript execution
Thread 4 → User interaction

Multiple tasks run simultaneously.

Simple Thread Diagram

Operating System
↓
Process
↓

Thread 1

Thread 2
Thread 3
Thread 4

Key Points for Exams

- Thread is the **smallest execution unit**
- Multiple threads can exist in one process
- Threads share process resources
- Two types: **User-level threads and Kernel-level threads**

Short Conclusion

A thread is the smallest unit of execution within a process. Multiple threads can run concurrently within the same process while sharing resources such as memory and files. Threads improve system performance and allow efficient multitasking in modern operating systems.

Types of Scheduling

1. Definition : Scheduling is the method used by the operating system to decide **which process should get the CPU and when**.

There are two main types of CPU scheduling.

1. Preemptive Scheduling

Definition : In **Preemptive Scheduling**, the operating system can **interrupt a running process** and allocate the CPU to another process.

This usually happens when:

- a higher priority process arrives
- the time quantum expires
- a shorter job becomes available

Characteristics

Feature	Description
CPU interruption	Allowed
Process switching	Frequent
Response time	Faster
Complexity	More complex implementation

Examples of Preemptive Scheduling

Algorithm
Round Robin
Shortest Remaining Time Next (SRTN)
Preemptive Priority Scheduling

Example

Suppose:

Process	Priority
P1	3
P2	1

If P1 is running and **P2 arrives with higher priority**, the OS interrupts P1 and runs P2.

2. Non-Preemptive Scheduling

Definition: In **Non-Preemptive Scheduling**, once a process starts executing, it continues until it **completes execution or moves to waiting state**.

The CPU is **not taken away** from the running process.

Characteristics

Feature	Description
CPU interruption	Not allowed
Process switching	Less frequent
Implementation	Simple
Response time	Slower

Examples of Non-Preemptive Scheduling

Algorithm
First Come First Serve (FCFS)
Shortest Job First (SJF)
Non-preemptive Priority Scheduling

Example

If P1 starts execution, it will **finish completely** before P2 gets CPU.

Comparison of Preemptive and Non-Preemptive Scheduling

Feature	Preemptive Scheduling	Non-Preemptive Scheduling
CPU interruption	Allowed	Not allowed
Response time	Faster	Slower
Implementation	Complex	Simple
Examples	RR, SRTN	FCFS, SJF

Thread Scheduling

1. Definition : Thread Scheduling is the process of selecting which thread should run on the CPU when multiple threads are available.

Since a process can contain multiple threads, the operating system must schedule **threads instead of processes** in multithreaded systems.

Types of Thread Scheduling

Thread scheduling depends on how threads are managed.

Type	Description
User-Level Thread Scheduling	Managed by user-level thread library
Kernel-Level Thread Scheduling	Managed by operating system kernel

1. User-Level Thread Scheduling

Threads are managed by **user-level libraries** without kernel involvement.

Characteristics:

Feature	Description
Managed by	User thread library
Kernel awareness	OS does not know about threads

Switching speed	Very fast
Blocking issue	Entire process may block

Examples:

- POSIX threads
- Java threads

2. Kernel-Level Thread Scheduling

Threads are managed directly by the **operating system kernel**.

Characteristics:

Feature	Description
Managed by	Operating system
Kernel awareness	OS manages threads
Performance	Slightly slower
Efficiency	Better CPU utilization

Examples:

- Windows threads
- Linux kernel threads

Simple Thread Scheduling Diagram

Process

Thread 1 → Ready

Thread 2 → Running

Thread 3 → Waiting

↓

Thread Scheduler

↓

CPU

Real-Time Scheduling

1. Definition: Real-Time Scheduling is a scheduling method used in real-time operating systems where tasks must be executed **within strict time deadlines**.

The scheduler ensures that **critical tasks are completed before their deadlines**.

Characteristics of Real-Time Scheduling

Feature	Description
Deadline-based execution	Tasks must complete on time
Priority-based scheduling	Critical tasks get priority
Deterministic behavior	Predictable response time
Low latency	Very quick response

Types of Real-Time Scheduling

Real-time scheduling is commonly divided into two types.

Type	Description
Hard Real-Time Scheduling	Missing a deadline causes system failure
Soft Real-Time Scheduling	Missing a deadline reduces performance but system continues

Examples of Real-Time Scheduling Systems

System	Application
Aircraft control systems	Flight control
Medical monitoring	Heart monitoring devices
Industrial robots	Automated manufacturing
Traffic signal control	Road traffic management

Simple Real-Time Scheduling Diagram

External Event

↓

Real-Time Scheduler

↓

High Priority Task

↓

Immediate Execution

Short Conclusion

Scheduling determines how the operating system allocates CPU time to processes and threads. Scheduling can be **preemptive or non-preemptive**, depending on whether the running process can be interrupted. In multithreaded systems, **thread scheduling** determines which thread runs next. Real-time scheduling ensures that critical tasks are executed within strict deadlines, which is essential for time-sensitive applications.

System Calls

1. Definition: A **System Call** is a mechanism through which a **user program requests services from the operating system kernel**.

User programs cannot directly access hardware resources such as CPU, memory, or devices. They must request these services using system calls.

Simple exam definition : A system call is an interface between a user program and the operating system that allows a program to request services from the kernel.

Purpose of System Calls

System calls allow user programs to interact with the operating system safely.

Main purposes:

Purpose	Explanation
Hardware access	Programs access hardware through OS
Resource management	OS manages CPU, memory, devices
Security	Prevents direct hardware manipulation
Communication	Enables process interaction

Basic Flow of System Calls

When a program needs a service, it makes a system call to the operating system.

User Program

↓

System Call Interface

↓

Operating System Kernel

↓

Hardware
The OS performs the requested operation and returns the result to the program.

Types of System Calls
System calls are categorized based on the services they provide.

Type	Purpose
Process Control	Manage processes
File Management	Manage files
Device Management	Control hardware devices
Information Maintenance	Obtain system information
Communication	Communication between processes
Protection	Security and access control

1. Process Control System Calls
These system calls manage the creation, execution, and termination of processes.
Operations include:

- Creating processes
- Terminating processes
- Executing programs
- Waiting for process completion

Common Process Management System Calls

System Call	Purpose
fork()	Creates a new process
exec()	Executes a new program
wait()	Waits for a child process to finish
exit()	Terminates a process
getpid()	Returns process ID

Explanation of Important Process System Calls
1. fork()
The **fork()** system call creates a **new process (child process)**.
The child process is a copy of the parent process.
Example:
Parent Process
↓
fork()
↓
Parent Process + Child Process

2. exec()
The **exec()** system call replaces the current process with a new program.
Example:
Process Running
↓
exec()
↓
New Program Loaded

3. wait()

The **wait()** system call causes the parent process to wait until the child process finishes execution.

Example:

Parent Process

↓

wait()

↓

Child Process Finishes

4. exit()

The **exit()** system call is used to terminate a process.

After termination, the operating system releases the resources used by the process.

Example of Process Creation Using System Calls

Parent Process

↓

fork()

↓

Child Process Created

↓

Child executes new program using exec()

↓

Parent waits using wait()

Advantages of System Calls

Advantage	Explanation
Secure hardware access	Programs cannot access hardware directly
Standard interface	Provides consistent OS services
Resource control	OS manages system resources efficiently

Disadvantages of System Calls

Disadvantage	Explanation
Overhead	System calls require kernel mode switching
Performance cost	Slight delay during system call execution

Simple System Call Diagram

Application Program

↓

System Call

↓

Operating System Kernel

↓

CPU / Memory / I/O Devices

Key Points for Exams

- System calls provide the **interface between user programs and the OS**
- Programs request services through **system call interface**
- Process-related system calls include **fork(), exec(), wait(), exit()**

- They enable **process creation, execution, and termination**

Short Conclusion

System calls are the interface between user programs and the operating system kernel. They allow programs to request system services such as process creation, file management, device control, and communication. Process management system calls like **fork(), exec(), wait(), and exit()** are used to create and manage processes in the operating system.